

智学引擎 ZhiXue Engine

最终项目报告

面向计算机课程学习的多智能体学习系统

报告日期：2026-06-27 | 文档类型：PDF，不超过 20 页

18

专职智能体协作

100/100

基础RAG hit@3

94/100

图谱RAG hit@3

>5min

长任务SSE不断连设计

这份报告按代码里的真实边界来写：前端负责学习工作台和流式交互，Java 负责用户、任务、鉴权和持久化，Python Agent 负责检索、规划、生成与评审。重点不是泛泛介绍“AI 平台”，而是说明 Context Engineering、Memory Engineering、RAG、Agent Runtime 和长任务链路具体如何落到工程实现。

报告内容来自 README、架构文档、部署文档、实验日志、RAG benchmark JSON，以及 Java/Python/前端源码中的模块命名与数据流。

报告范围

- 覆盖 2026-05-08 到 2026-06-27 的工程阶段、保留结论和回滚案例。
- 详细说明 context、memory、retrieval、runtime、resource generation、SSE 和用户 LLM Key 边界。
- 所有关键指标直接引用仓库内测试记录和 benchmark 报告。

一、项目完成情况

项目最终交付的是一个可运行的学习系统，而不是单独的聊天框。学习者可以从日常工作台进入问答、资源生成、错题复习、笔记、画像和知识图谱。Java 是唯一外部 API 入口，Python Agent 只作为内部智能体运行时存在，浏览器不会直接访问 Python 的 /internal/* 接口。

模块	用户看到的功能	代码中的落点
智能问答	多轮对话、图片问题、深度思考、联网搜索、逐字流式回复	ConversationController -> ConversationService -> HttpPythonConversationMessageClient -> TutorAgent
个性化学习	学习路径、掌握度分析、知识点弱项、每日学习任务	PersonalizedLearningContextService、ProfileAgent、EvaluationAgent、PathPlanningAgent、LearningPath*Service
资源生成	讲解文档、PPT、思维导图、练习、代码案例、视频资源	ResourceBundleWorkflow fan-out, Document/Slides/MindMap/Practice/Code/Video agents
错题复习	错题列表、复习会话、掌握状态、间隔复习	MistakeBookService、PracticeResultPersistenceService、mistake_record 相关迁移
笔记与资源库	AI 笔记、语义搜索、资源收藏、进度记录	NoteService、ResourceLibraryService、Note/Resource semantic search client
语音助手	悬浮入口、ASR/TTS、语音命令、页面上下文问答	VoiceController、VoiceGatewayService、VoiceRealtimeWebSocketHandler、VoiceContextRequest

我们保留下来的工程边界

- 浏览器只认 /api/*，所有内部智能体调用由 Java 加内部 Token 转发，避免前端绕过鉴权和审计。
- 长任务不靠同步 HTTP 等待，统一走 Redis Streams 入队、Python Worker 消费、Java 持久化事件、前端 SSE 订阅。
- 生成资源必须带 LLM provenance，Java 校验后才签发 downloadUrl，避免伪生成资源进入用户可下载链路。
- 正式用户任务必须使用设置页保存的用户 LLM Key 和模型，系统 Key 只用于测试账号、自动化测试和联调。

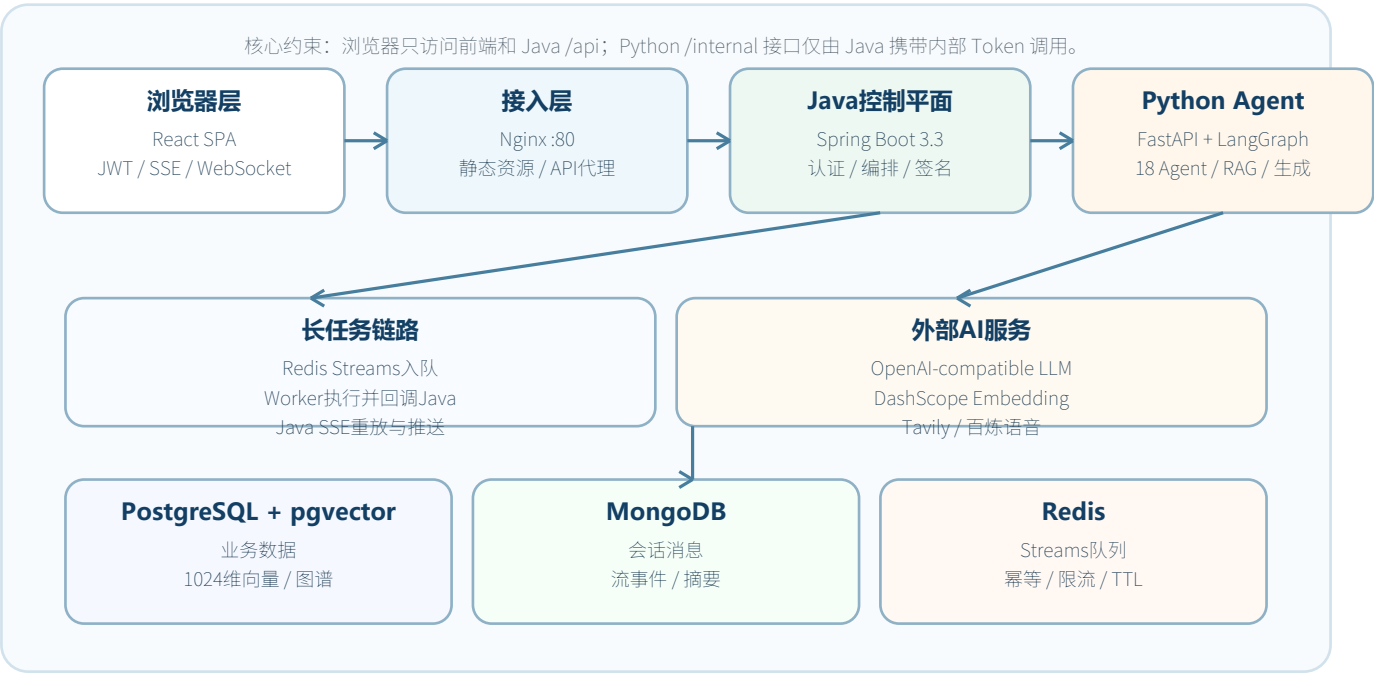
二、阶段重要成果

阶段	时间	关键成果	验证或指标
基础自举与部署	05-08 到 05-14	修复数据库初始化、向量数据恢复、登录链路；补齐 README、架构文档和部署文档；确认 Docker Compose 六服务拓扑。	注册/登录 200；向量数据恢复无孤儿引用；健康检查通过。
学习闭环与资源生成	05-13 到 05-17	修复资源生成、练习判题、学习评估、错题本、前端性能拆分和 QNA 流式状态；建立自动沉淀错题与复习调度。	前端 tsc/build 通过；定向 Python/Java 测试通过；错题 API 恢复 200。
安全和契约加固	05-17 到 06-20	移除硬编码 Key；Java 到 Python 内部 Token；数据端口 loopback；SSE 事件契约与 LLM 错误结构化透传。	密钥 grep 无匹配；internal token 测试通过；Java/Python/前端健康无退化。
RAG 与 Graph RAG	05-24 到 06-14	四路 RAG、纯规则 QueryClassifier、图谱 intent、Graph 扩展、动态 RRF、direct evidence Top5 保护和 benchmark 固化。	基础 100 题 hit@3=100/100；图谱 100 题 hit@3=94/100；channelErrorCount=0。

阶段	时间	关键成果	验证或指标
体验与长任务	06-20 到 06-23	资源生成失败详情、单资源重试、QNA 多 Agent 工作区、UI/UX 统一、资源工作区折叠和热更新。	前端 tsc/build 与 vitest 通过；容器前端 / 返回 200；健康检查无退化。
上下文工程与收尾	06-25 到 06-27	动态 Token 预算、Prompt 缓存、元数据注入、结构感知压缩、语义重排序，以及资源语义搜索和工作区职责收敛。	34/34 相关测试通过；RAG 回归通过；最新实验日志记录全链路验证通过。

阶段推进遵循“指标改善才保留”的原则。典型案例是 2026-05-24 的 Grep 批量 ILIKE 与 trigram 索引实验：虽然 Grep 通道变快，但端到端平均延迟和质量变差，因此回滚代码和数据库索引，仅保留 benchmark 脚本与报告。

三、系统架构



架构原则	实现方式
Java 唯一入口	浏览器只访问前端和 /api/*；Python /internal/* 仅由 Java 携带 X-Zhixue-Internal-Token 调用。
关注点分离	前端渲染和流式解析；Java 负责编排、鉴权、持久化；Python 负责 AI 推理、RAG 和生成。
事件驱动	长任务经 Redis Streams 入队，Python Worker 消费，Java 持久化事件并通过 SSE 重放和推送。
契约先行	SSE wire format 固定为 event/data，新增事件需同步 contracts/sse-events.schema.json、Java 转发和前端解析。
可回溯	traceld、event_seq、audit_log、conversation_stream_events 支撑问题定位和断线续传。

核心链路

- 对话流：浏览器 POST 到 Java 对话流接口，Java 代理 Python /internal/smart-engine/stream，Python SSE 实时返回，Java SseEmitter 转发给浏览器。
- 长任务：Java 创建任务并写入 Redis Stream，Python Worker 执行并回调 Java，前端订阅任务 SSE 获取 progress、resource_file、done 和 error。

- 资源下载：Python 生成资源后写入共享 sandbox，Java 校验 provenance 后签发短 TTL downloadUrl。

四、RAG 检索与知识图谱成果

检索系统从单一向量召回演进为 Grep、Vector、Graph、Web 四路并行召回，再通过加权 RRF 融合。QueryClassifier 使用纯规则分类，避免为检索策略选择引入额外 LLM 成本。

通道	技术实现	作用
Grep	FMM 前缀 Trie、术语词典、同义词扩展、三阶段回退	保障精确术语和短语题命中
Vector	DashScope text-embedding-v4、pgvector 1024 维、知识/资源双表检索	提供语义相关召回
Graph	wiki_link 1-hop/2-hop、PageRank、社区加权、direct evidence 保护	补齐多跳关系和前置路径证据
Web	Tavily opt-in、时间敏感查询注入当前日期、8 秒超时	按用户开关补充实时信息
RRF 融合	k=60, grep/vector/graph/web 动态权重, Graph intent 调整 0.5 到 1.8	减少单通道失误并稳定 Top5

测试集	hit@1	hit@3	平均延迟	P95 延迟	通道错误
基础 RAG 100 题	100/100	100/100	861.07 ms	972.03 ms	0
图谱 RAG 100 题	56/100	94/100	652.38 ms	765.35 ms	0

- 基础 100 题达成 hit@3=100/100，说明普通知识问答召回稳定。
- 图谱 100 题达成 hit@3=94/100，overallPass=true，并保留 primaryTop5、evidenceNodeRecallTop5、completeEvidenceTop5 等诊断字段。
- 2026-06-27 修复资源语义搜索 hybrid shortcut，direct vector 与 hybrid RAG 合并排序，避免低分 hybrid RRF 短路高分向量结果。

五、Context Engineering

项目里的上下文工程不是简单把历史消息拼到 prompt 后面。我们把前端当前页面、Java 侧 30 天学习信号、Python 侧会话摘要和检索证据分层处理，最后由 SnapshotBuilder 生成 SystemSnapshot，再注入每个 Agent 的 system prompt。这样做的原因是：不同任务需要的上下文不同，资源生成、评估和问答不能共享同一坨无结构文本。

环节	具体实现	进入 prompt 的内容
前端上下文	ConversationMessageStreamRequest.voiceContext, VoiceContextRequest.normalizedMap()	pageType、currentPath、noteExcerpt、weakPointsSummary、resourceResultSummary、confirmedSlideOutline 等页面信号。
Java 聚合	PersonalizedLearningContextService.buildContext(userId)，默认聚合近 30 天数据	薄弱知识点、练习正确率、评估任务、错题统计、待复习错题、资源使用偏好。
Python 快照	SnapshotBuilder.build() 合并 profile、profileAnalysis、learningContext	currentCourse、currentChapter、studentLevel、knowledgeGaps、preferredStyle、recentMistakes、conversationLength、wikiPagesCount。
Prompt 渲染	SnapshotBuilder.render_prompt_context(SystemSnapshot)	以“当前上下文”块注入课程、进度、薄弱点、偏好、最近错误和会话指标。

环节	具体实现	进入 prompt 的内容
检索证据	evidence_formatter 注入 channel、score、hop、metadata	让模型区分精确匹配、语义命中、图谱扩展和联网证据，减少证据顺序造成的 lost-in-middle。

优化点	落地位置	解决的问题
动态 Token 预算	runtime/token_budget.py, TokenBudgetAllocator	按模型窗口分配 evidence/history/tool 输出预算，Claude Opus 预算从 1200 提升到 71100。
Prompt 缓存	LLM 调用层对 Claude/GPT-4 的 system 与长证据标记缓存	重复查询时复用稳定上下文，成本预期降低 90%。
结构感知压缩	runtime/structure_aware_compactor.py, 被 AgentCoreLoop 调用	JSON 不再硬截断，代码保留函数签名，普通文本保留关键句。
查询改写防漂移	retrieval/services.py 的 QueryRewriteService	显式主题已足够具体时，不再把课程/画像上下文强塞到 rewrittenQuery 前面。

一个具体例子：用户在知识图谱页问“监督学习和深度学习的关系”，前端会带上当前页面和知识点；Java 只补 30 天内真实学习信号；Python 的 QueryClassifier 判断是否需要 Graph intent；SnapshotBuilder 只把画像、弱项和当前主题整理成短结构；检索证据再由 formatter 带元数据进入 TutorAgent。

六、Memory Engineering

记忆系统分成“对话记忆”和“学习记忆”两类。对话记忆适合放 MongoDB，保留原始消息和结构化摘要；学习记忆放 PostgreSQL，和用户、任务、错题、计划、知识图谱绑定。这样设计后，问答可以恢复上下文，个性化任务也能拿到真实行为数据，而不是每次都依赖模型临时猜测。

记忆层	主要文件或表	写入/读取方式
原始会话	conversation_message_store.py; Mongo conversation_messages	用户消息、assistant 回复、reasoningContent、imageUrls 按 conversationId 保存，历史消息接口分页读取。
结构化摘要	conversation_summary_store.py; Mongo conversation_summaries	ConversationCompactor 抽取 topicFocus、knownGaps、unresolvedQuestions、preferredHelpStyle，供长对话压缩后继续使用。
学习画像	profile_store.py; app.user_profile_current / user_profile_snapshot	ProfileAgent 与练习/评估结果更新画像，保留当前态和历史快照。
知识图谱	knowledge_graph_store.py; app.learner_knowledge_node / learner_knowledge_edge	按 canonical_key、topic、mastery_score、node_status 维护知识掌握状态和依赖边。
学习计划	learning_plan_store.py; learning_plan 相关表	PathPlanningAgent 输出路径后持久化版本，工作台和后续任务可读取。
练习与错题	practice_store.py、PracticeResultPersistenceService、MistakeBookService	判题结果写入 practice_submission，错误通过触发器 /upsert 进入 mistake_record，SM-2 复习调度继续更新。

记忆如何进入一次任务

• PERSONALIZED_LEARNING 任务提交后，SmartEngineOrchestratorService 会调用 PersonalizedLearningContextService，把画像、知识节点、练习、评估、错题和资源偏好塞进 invocationParams。

- Python Supervisor 收到 EngineStreamRequest 后先构建 SystemSnapshot，再按 serviceType 执行 profile -> evaluation -> query_rewrite -> retrieval -> path_planning -> resource_push -> critic。
- 对话任务则更多依赖 conversation_messages、conversation_summaries 和本轮 voiceContext，避免把长期计划数据过量注入普通问答。
- 画像和知识图谱不是只展示给前端，后续资源推荐、路径规划和错题复习都会反过来消费这些 memory。

七、Agent Runtime 与资源生成

PythonAgentSupervisor 是智能体运行时的入口。它不直接把所有请求丢给同一个大模型，而是先通过 serviceType、QueryClassifier 和 route template 解析 RoutePlan，再按顺序或 LangGraph 并行工作流调度不同 Agent。这个运行时承担了路由、快照、事件序号、取消、错误分类和资源 provenance 校验。

链路	RoutePlan / Agent 顺序	说明
普通问答	tutor 或 query_rewrite -> retrieval -> tutor	QueryClassifier 根据小聊天、追问、图片题、当前信息等决定是否检索、是否联网、是否图谱增强。
个性化学习	profile -> evaluation -> query_rewrite -> retrieval -> path_planning -> resource_push -> critic	先整理画像和诊断，再规划路径，最后推荐资源并做 Critic 审核。
练习判题	practice -> judge -> profile	PracticeAgent 出题，JudgeAgent 判题，结果写回画像和错题记忆。
资源包生成	ResourceBundleWorkflow fan-out	DOCUMENT/SLIDES/MINDMAP/QUIZ/CODE/VIDEO 映射到不同 Agent，并行生成后汇总资源和失败项。
视频生成	query_rewrite -> retrieval -> video_generator	服务端生成脚本/资源事件，浏览器侧负责本地渲染和下载。

运行时机制	代码落点	项目里解决的问题
AgentCoreLoop	runtime/agent_core_loop.py	支持工具调用、多轮迭代、权限策略、hook chain、恢复引擎和结构感知工具输出压缩。
ResourceBundleWorkflow	runtime/resource_bundle_workflow.py	LangGraph StateGraph 并行调度资源 Agent，收集 resource_results 和 resource_failures。
Critic 软门禁	review agents + resource bundle workflow	PASS_WITH_ISSUES、NEEDS_MINOR_REVISION 等不再误判为整体失败，硬失败保留失败原因。
Provenance 校验	runtime/provenance.py, Java 回调校验，前端展示读取	所有 resource_file 必须带 provider、model、agentName、evidencelds、fallback 等来源信息。
结构化错误	llms/errors.py、PythonAgentStreamException、前端 ConversationStreamError	限流、额度、鉴权、模型不存在、上下文过长等错误不会被粗暴显示成“系统开小差”。

八、全栈实现与可靠性

层级	关键实现	风险控制
前端	React Router、fetch + ReadableStream SSE、Markdown/Mermaid/KaTeX、语音面板、资源工作区托盘	侧栏只保留本轮产物入口，完整编辑/下载/预览收敛到资源包详情页。
Java	Spring Security、SseEmitter、Redis Streams 入队、任务状态机、下载签名、用户 LLM 设置	JWT 和内部 Token 分流；task event 持久化；downloadUrl 短 TTL。

层级	关键实现	风险控制
Python	FastAPI、LangGraph、RAG 四通道、AgentCoreLoop、ResourceBundleWorkflow、SmartEngine Worker	通道级降级、结构化 LLM 错误、sandbox 清理、任务取消标记。
数据	PostgreSQL app/rag/storage 三 schema、Mongo 会话集合、Redis Streams/TTL	向量维度固定 1024；已有环境走 migrations，不通过重建数据容器改 schema。
接入	Nginx SPA fallback、/api 代理、SSE 缓冲关闭、1800 秒读取超时	长任务不被代理截断；WebSocket 使用单独超时和鉴权处理。

关键可靠性设计

- SSE replay/live 去重串行化，前端断线重连后可从持久化事件恢复。
- Redis SETNX + TTL 用于幂等、限流和取消标记，避免重复提交和无限缓存。
- LLM 错误分类保留 rate limit、quota、auth、permission、model invalid、context too long 和 timeout 等可处理信息，未知内部异常仍泛化展示。
- 当前联调/演示环境遵守热更新边界：只允许 docker cp 同步静态文件、JAR 或 Python 源码，禁止 build/recreate。

九、验证结果与质量状态

项目没有用“看起来好了”作为验收标准。每次改动都在 docs/experiment_log.md 追加一行，写清楚改了什么、跑了什么、指标怎么变、最后保留还是回滚。这份报告也只改文档产物，不改业务代码、不改 Docker 拓扑、不改数据库 schema。

验证项	命令或方式	当前记录
全链路功能	py -m pytest tests/ -v	6 通过，3 个既有 collection warning。
RAG 回归	py -m pytest python-agent/tests/ -k rag -v	18 通过，hits@3 指标满足 >= 90%。
Python Agent	test_server、test_vector_searcher、test_retrieval_services	定向测试通过，资源语义搜索回归保持通过。
Java 后端	cd project && mvn test	最新 2026-06-27 日志记录通过。
前端构建	cd frontend && npx tsc --noEmit && npx vite build	通过，仅保留既有 Browserslist/大 chunk 警告。
运行态健康	docker compose ps；curl /api/health	全服务 Up，Java /api/health UP。

质量边界	状态
SSE 协议	未改变 event/data wire format，前端、Java、Python 仍按契约对齐。
数据库 schema	本报告未修改 schema；既有项目约束要求通过 migrations 演进。
向量维度	固定 1024，报告和脚本未触碰。
Docker 拓扑	报告生成不涉及 compose 拓扑，也不执行 rebuild/recreate。
安全密钥	报告不包含真实 .env 密钥，不读取或输出用户 API Key。

十、AI 协同开发复盘



项目后半程基本按工程问题拆线推进，而不是按“让 AI 写一段代码”推进。下面这张表按实际工作内容复盘，列出每条线负责的模块、关键文件和最后沉淀下来的结果。这些角色不是组织架构，而是我们在迭代中反复处理的工程职责。

工程线	具体负责的事	关键落点
Context Engineer	控制一次请求带多少上下文、从哪里取上下文、如何避免画像/课程信息污染显式问题；把页面信号、30 天学习信号、检索证据和系统快照拆开处理。	PersonalizedLearningContextService、VoiceContextRequest、SnapshotBuilder、evidence_formatter、QueryRewriteService。
Memory Engineer	把短期会话、结构化摘要、画像、知识图谱、学习计划、练习和错题分层存储；保证问答能恢复上下文，个性化学习能读取真实行为数据。	conversation_message_store.py、conversation_summary_store.py、profile_store.py、knowledge_graph_store.py、learning_plan_store.py、practice_store.py。
Retrieval Engineer	把 Grep、Vector、Graph、Web 四路召回接到同一个融合层；修复 graph intent、direct evidence Top5、hybrid shortcut 和低证据诊断。	query_classifier.py、services.py、hybrid_retriever.py、graph_expander.py、rrf_fusion.py、semantic_reranker.py。
Agent Runtime Engineer	维护 PythonAgentSupervisor、AgentCoreLoop 和 ResourceBundleWorkflow；处理路由、工具调用、取消、结构化错误、资源失败详情和 provenance。	supervisor.py、agent_core_loop.py、resource_bundle_workflow.py、provenance.py、llms/errors.py、events.py。
Control Plane Engineer	保证 Java 唯一入口、JWT 与内部 Token 分流、任务状态机、SSE replay、下载签名和用户 LLM 设置边界。	ConversationService、SmartEngineOrchestratorService、SseEmitterService、TaskStateMachineService、UserLlmSettingsService。
Frontend Experience Engineer	处理流式回复、资源工作区托盘、资源包详情页、错误提示、青蓝教育 SaaS 视觉层和移动端折叠状态。	LearningStudioDemoPage、QnaAgentWorkspacePanel、resourceGenerationStore、MultiAgentResourceGenerationPage、index.css。

复盘维度	做法	收益
任务拆分	主 Agent 承担端到端实现；RAG 和前端关键改动中使用 verifier、bug hunter 等子代理复核。	减少遗漏契约、边界和回归风险。
指标驱动	每轮以测试、benchmark、健康检查和日志为准；指标变差则回滚。	05-24 RAG 性能实验及时回滚，保留数据而不保留退化代码。
手术级修改	只动问题相关文件，避免顺手重构；热更新场景不重建容器。	降低演示环境中断概率，也便于追踪责任边界。
跨层契约	SSE schema、内部 Token、downloadUrl provenance、用户 LLM Key 边界同步检查。	避免前端、Java、Python 各自“局部正确”但联调失败。
文档化	README、架构文档、部署文档、experiment_log 与代码同步更新。	为答辩、交接和复盘提供可验证证据链。

经验教训

- RAG 优化必须同时看质量和延迟，单通道变快不等于端到端收益。
- 长任务体验的关键不只是生成成功，还包括失败详情、局部重试、断线恢复和最终资源入口。

- 用户 LLM Key 是正式任务边界，环境 Key 只适合测试和自动化联调。
- AI 代理适合加速实现和审查，但最终仍要回到自动化测试、真实 benchmark 和运行态健康。

十一、总结与后续方向

截至 2026-06-27，智学引擎已经不是单点 demo，而是一套有入口、有记忆、有检索、有资源生成、有评估闭环、有运行指标的全栈项目。基础 RAG hit@3=100/100，图谱 RAG hit@3=94/100，长任务走 Redis Streams + SSE，用户 LLM 配置、内部鉴权和资源 provenance 都已经有了清晰边界。

后续方向	建议
RAG 深化	继续扩大图谱题型集，针对 lowEvidenceRecordsByIntent 做数据和排序双向修复。
端到端体验	补齐浏览器自动化覆盖，持续检查登录、SSE、下载和 >5min 任务不断连。
成本控制	观察 Prompt 缓存和 TokenBudgetAllocator 在真实用户任务中的成本曲线。
运维发布	把热更新经验沉淀为维护窗口发布流程，区分演示热更新与全新部署。
AI 协同流程	保留 verifier/bug hunter 复核机制，并将关键指标纳入自动化报告。

资料来源

- README.md、docs/架构文档.md、docs/deployment.md、docs/experiment_log.md。
- python-agent/reports/rag_100_phase3_20260614.json。
- python-agent/reports/graph_rag_100_phase3_pass_20260614.json。